

Enterprise AI Architect Learning Path v3 – Phase Guide 4

Enterprise AI Architect Learning Path v3 – Phase Guide 4

Agent Orchestration

Purpose of This Phase

Phases 2 and 3 established a production-quality knowledge platform capable of retrieving and evaluating information.

Phase 4 introduces the next major evolution:

Moving from answering questions to performing work.

This is where the learning path transitions from Retrieval-Augmented Generation (RAG) systems into Agentic AI systems.

An agent is not simply a chatbot. An agent is a software component that can:

- Maintain state
- Make decisions
- Use tools
- Execute workflows
- Coordinate multiple operations
- Collaborate with humans

This phase focuses on building those capabilities while maintaining enterprise-grade controls and governance.

Why This Phase Matters

Many organizations have already built RAG systems.

Far fewer have successfully built agent systems that are:

- Reliable
- Traceable
- Stateful
- Governed
- Recoverable

This phase introduces the architectural patterns required for enterprise agent systems.

The objective is not:

Build an autonomous AI.

The objective is:

Build a controllable AI system that can safely participate in business processes.

Phase Objectives

Upon completion of this phase, you should be able to:

- Explain agent architectures
 - Design stateful workflows
 - Build Semantic Kernel-based agents
 - Build LangGraph-based workflows
 - Implement human-in-the-loop controls
 - Manage agent memory
 - Persist agent state
 - Recover interrupted workflows
 - Evaluate orchestration approaches
 - Design enterprise agent architectures
-

Architect Mindset

The most important lesson of this phase:

Agents are workflows, not personalities.

Avoid thinking:

- "What should the agent say?"

Instead think:

- What state does it maintain?
- What tools can it access?
- What decisions can it make?
- What decisions require human approval?
- What actions must be auditable?

Enterprise architecture is about controlling behavior, not maximizing autonomy.

Primary Resources

Required

R8

Semantic Kernel Documentation

Focus Areas:

- Kernels
 - Plugins
 - Functions
 - Filters
 - Process Framework
-

R9

Introduction to Semantic Kernel

Focus Areas:

- Microsoft-first orchestration patterns
 - Middleware concepts
 - Enterprise integration
-

R10

LangGraph Documentation

Focus Areas:

- StateGraph
 - Persistence
 - State management
 - Durable workflows
-

R11

LangGraph + Azure AI Foundry

Focus Areas:

- Microsoft ecosystem integration
 - Hosted agent environments
-

R18

Generative AI with LangChain

Focus Areas:

- Agent patterns
 - Long-running workflows
 - Multi-step orchestration
-

R21

LangChain Academy

Focus Areas:

- Agent design
 - Agent evaluation
 - Graph-based workflows
-

Core Technical Skills

1. Tool Calling

Agents are most valuable when connected to capabilities beyond text generation.

Examples:

- Search
- Retrieval
- Calculations
- APIs
- Databases

An agent should be viewed as a decision-maker that uses tools rather than a language model that occasionally generates text.

2. State Management

The system must understand:

Current State

Examples:

- Session context
 - Campaign state
 - Active objectives
-

Historical State

Examples:

- Previous actions
 - Prior decisions
 - User interactions
-

Persistent State

Examples:

- Long-term memory
- Saved workflows
- Stored decisions

State management becomes one of the most important architectural concerns in agent design.

3. Human-in-the-Loop (HITL)

Enterprise systems rarely permit unrestricted execution.

Learn how to implement:

Approval Gates

Examples:

- Major world changes
 - Workflow completion
 - Publishing decisions
-

Review Steps

Examples:

- Content approval
 - Rules validation
 - Decision verification
-

Escalation Paths

When confidence is low:

- Stop
- Ask questions
- Request human input

4. Workflow Design

Move beyond simple prompts.

Design:

- Steps
- States
- Transitions
- Decision paths

Agents should be modeled as workflows rather than conversations.

5. Durable Execution

The system should survive:

- Restarts
- Time gaps
- Failures
- Interruptions

Long-running workflows are a defining characteristic of enterprise agents.

Semantic Kernel vs LangGraph

One of the primary learning goals of this phase is understanding architectural tradeoffs.

Semantic Kernel

Best viewed as:

AI middleware

Strengths:

- Microsoft ecosystem alignment
- Plugin architecture
- Enterprise integrations
- Familiar patterns for .NET developers

Good for:

- Tool orchestration
- Enterprise applications

- Business process integration
-

LangGraph

Best viewed as:

Stateful workflow orchestration

Strengths:

- State machines
- Persistence
- Human checkpoints
- Long-running workflows

Good for:

- Complex workflows
 - Multi-step agent processes
 - Durable state management
-

Architect Outcome

By the end of this phase you should be able to justify:

- Why Semantic Kernel was chosen
- Why LangGraph was chosen
- When both should coexist

The goal is not framework loyalty.

The goal is architectural reasoning.

Phase Project

Virtual GM v1

This phase introduces the first true agent platform.

The Virtual GM serves as a practical example of enterprise orchestration concepts.

Functional Requirements

Session State

Maintain:

- Campaign information
- Session context
- Character state
- World state

Across multiple interactions.

Rules Retrieval

Leverage the RPG Catalog platform.

The agent should retrieve information rather than rely on model memory.

Encounter Support

Provide structured assistance for:

- Encounters
 - Story progression
 - Rule interpretation
-

Continuity Management

Maintain consistency between:

- Past events
 - Current decisions
 - Future actions
-

Human Approval

Certain actions require explicit authorization.

Examples:

- Major world changes
 - Canon modifications
 - Story-critical events
-

Architecture Deliverables

ADR-010

Agent Architecture Strategy

Document:

- Agent responsibilities
 - State management strategy
 - Tool strategy
-

ADR-011

Framework Selection

Document:

- Semantic Kernel findings
 - LangGraph findings
 - Final architectural rationale
-

ADR-012

Human Governance Model

Document:

- Approval requirements
 - Escalation rules
 - HITL design
-

Portfolio Artifacts

Workflow Diagram

Demonstrate:

Player Input

↓

State Review

↓

Rules Retrieval

↓

Decision Logic

↓

Approval Check

↓

Response

State Model Diagram

Document:

- Session state
 - Campaign state
 - Memory model
-

Agent Design Document

Describe:

- Responsibilities
 - Boundaries
 - Tools
 - Constraints
-

Architecture Comparison Report

Compare:

- Semantic Kernel
- LangGraph

and justify chosen approaches.

Suggested Sprint Progression

Sprint 1 — Agent Foundations

Goals:

- Learn agent concepts
- Explore frameworks

Done When:

- Basic orchestration prototype exists
-

Sprint 2 — State Model

Goals:

- Design memory structures
- Design persistence model

Done When:

- State architecture documented
-

Sprint 3 — Tool Integration

Goals:

- Connect RAG platform
- Enable retrieval workflows

Done When:

- Agent accesses RPG catalog
-

Sprint 4 — Human Approval

Goals:

- Implement approval checkpoints
- Add governance controls

Done When:

- HITL workflow functions
-

Sprint 5 — Workflow Hardening

Goals:

- Recovery
- Persistence
- Reliability

Done When:

- Workflow survives interruptions
-

Sprint 6 — Framework Evaluation

Goals:

- Compare frameworks
- Produce recommendations

Done When:

- ADRs completed
-

Sprint 7 — Virtual GM v1

Goals:

- Complete agent platform

Done When:

- End-to-end workflow operational
-

Common Pitfalls

Pitfall 1

Treating Agents Like Chatbots

Agents are workflow systems.

A conversation is only one interface.

Pitfall 2

No State Strategy

Many agent systems fail because state management was never architected.

State should receive as much attention as prompting.

Pitfall 3

Unlimited Autonomy

Enterprise systems require controls.

Human approval exists for a reason.

Pitfall 4

Framework Worship

Neither Semantic Kernel nor LangGraph is universally superior.

Architectural requirements determine framework selection.

Pitfall 5

Ignoring Recovery Scenarios

Ask:

What happens if the process crashes halfway through?

Good agent architectures answer this question.

Enterprise Translation Layer

The Virtual GM demonstrates patterns applicable to many enterprise systems.

	≡ Virtual GM Capability	≡ Enterprise Equivalent
1	Campaign State	Customer Context
2	Session Memory	Case Management
3	Rules Retrieval	Policy Retrieval
4	Approval Gate	Human Review Workflow
5	Encounter Planning	Process Orchestration
6	World State Changes	Business State Changes

The architecture is transferable even when the domain changes completely.

Definition of Done

Phase 4 is complete when:

- Agent workflows are operational
 - State management is implemented
 - Persistence strategy exists
 - Human approval workflows exist
 - Retrieval integration functions
 - Workflow diagrams are complete
 - Framework comparison is documented
 - ADRs are completed
 - Virtual GM v1 operates end-to-end
-

End-of-Phase Self-Assessment

You are ready for Phase 5 when you can confidently answer:

- Can I explain the difference between an agent and a chatbot?
- Can I design stateful workflows?
- Can I justify Semantic Kernel versus LangGraph?
- Can I implement human-in-the-loop controls?
- Can I persist and recover workflow state?
- Can I architect an enterprise agent platform rather than merely prompt an LLM?

If all answers are "yes," proceed to **Phase 5 – MCP & Tool Integration**.